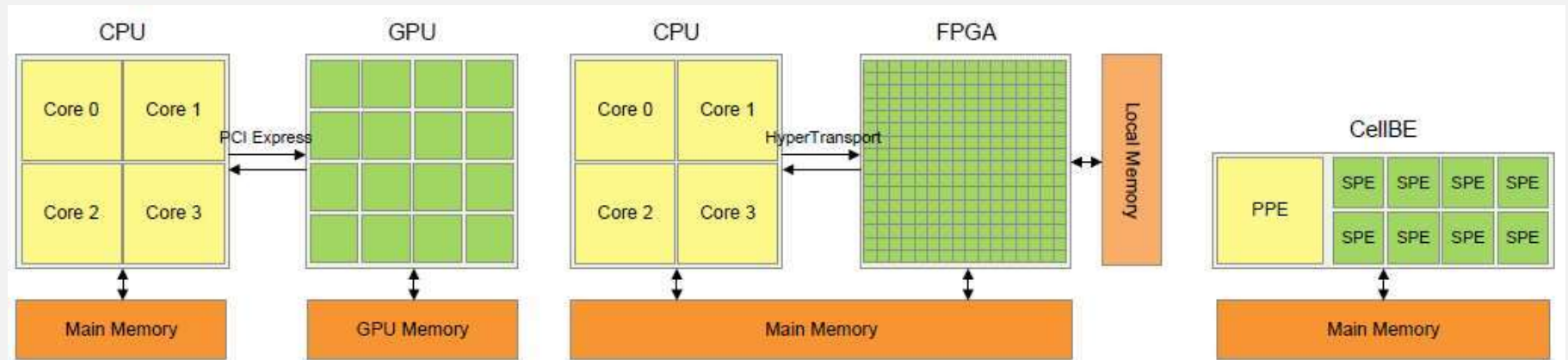


Computación heterogénea

Introducción

Más de un procesador trabajando conjuntamente.

Distintas arquitecturas de procesadores que se complementan.



Computación heterogénea

Introducción

Caso típico. Computación en GPU

Relación con procesador (CPU):

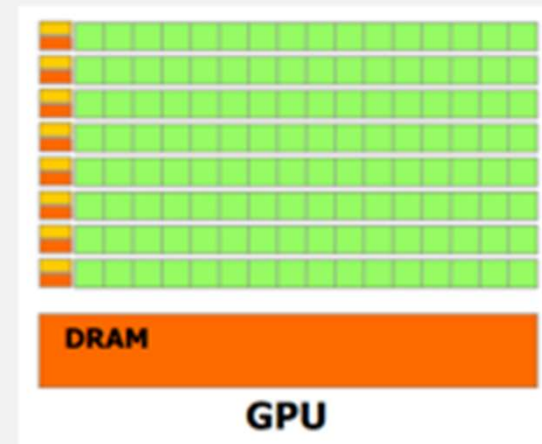
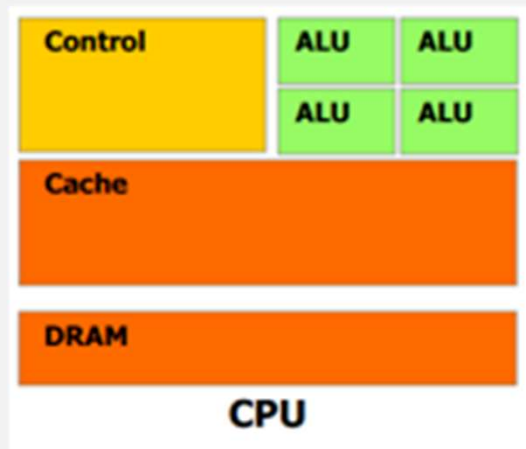
- No es posible reemplazar la CPU por una GPU. CPU organiza trabajo GPU.
- CPU formada por varios núcleos que comparten memoria.
- GPU consta de millares de núcleos pequeños, diseñados para manejar múltiples tareas simultáneamente de forma eficiente.



Computación heterogénea

Introducción

Visión general CPU - GPU



Computación heterogénea

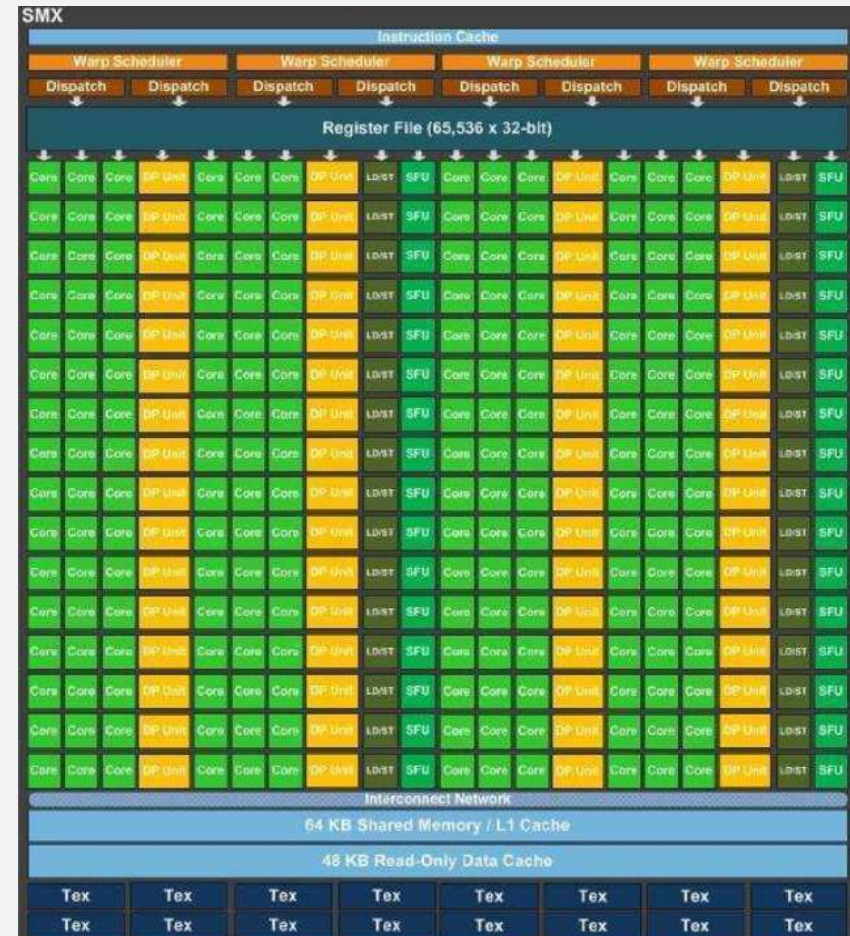
Introducción

Ejemplo GPU. Arquitectura Kepler.

192 núcleos (6 / grupo).

64 DP units.

65536 registros.



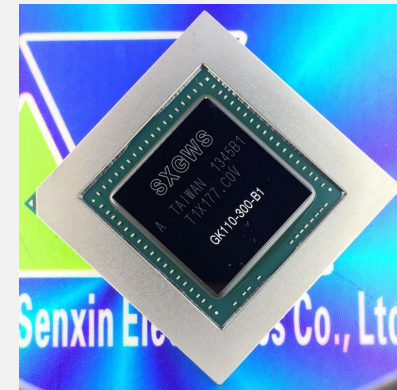
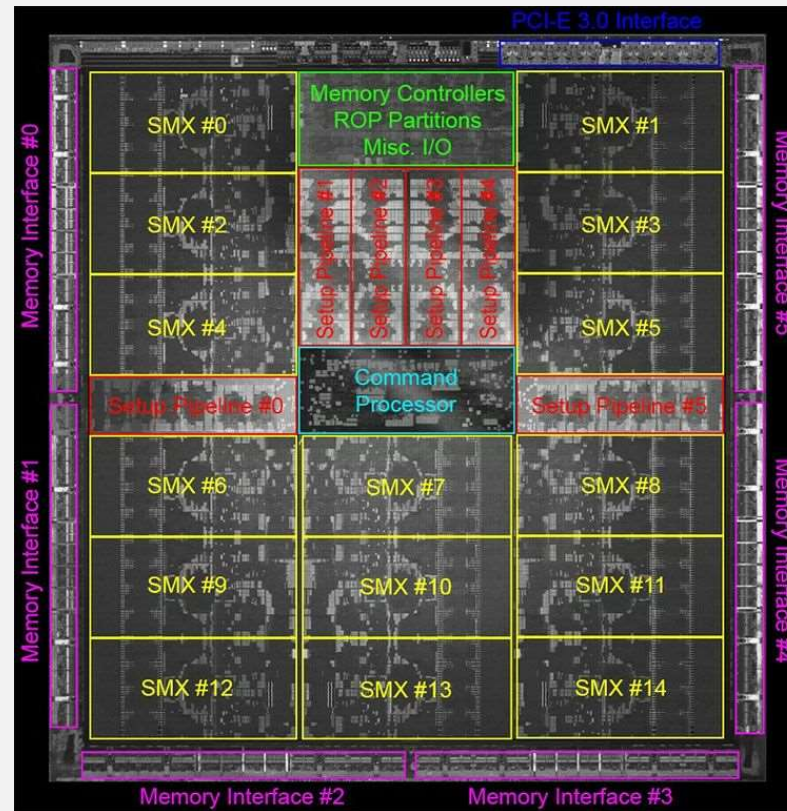
Computación heterogénea

Introducción



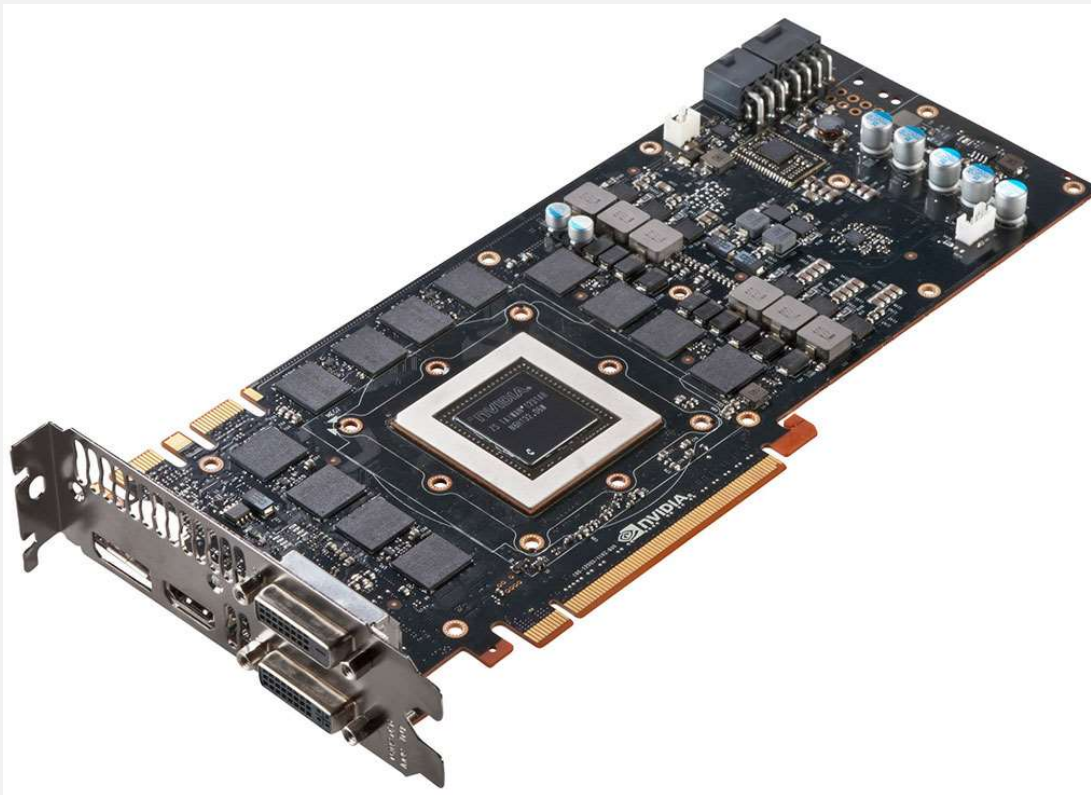
Computación heterogénea

Introducción



Computación heterogénea

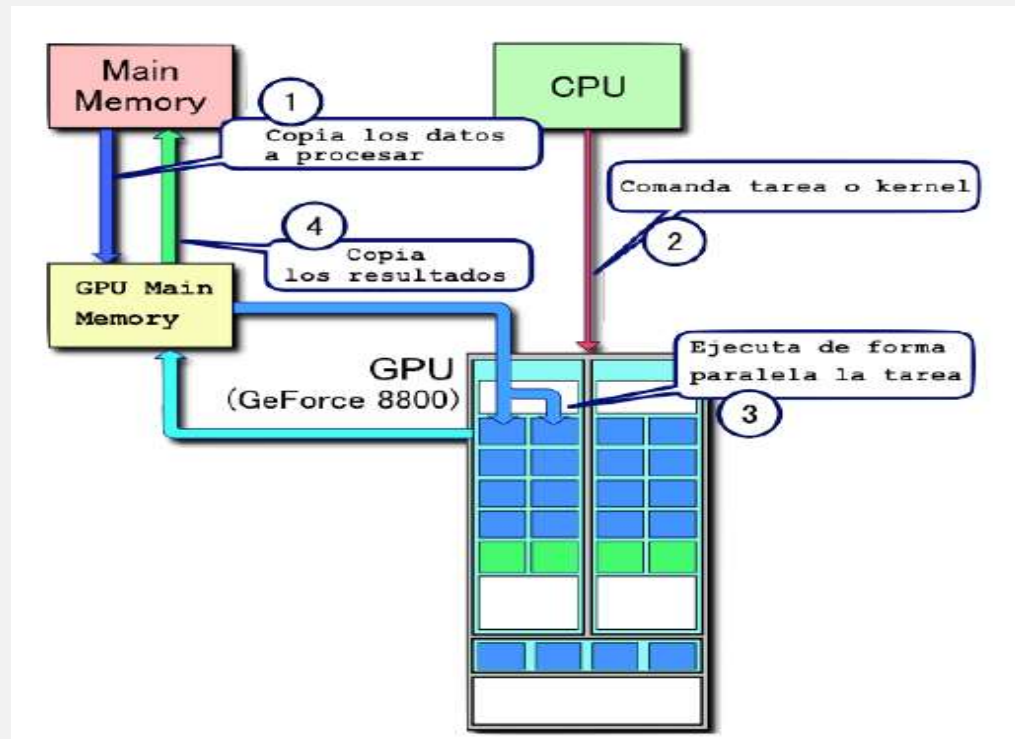
Introducción



Computación heterogénea

Introducción

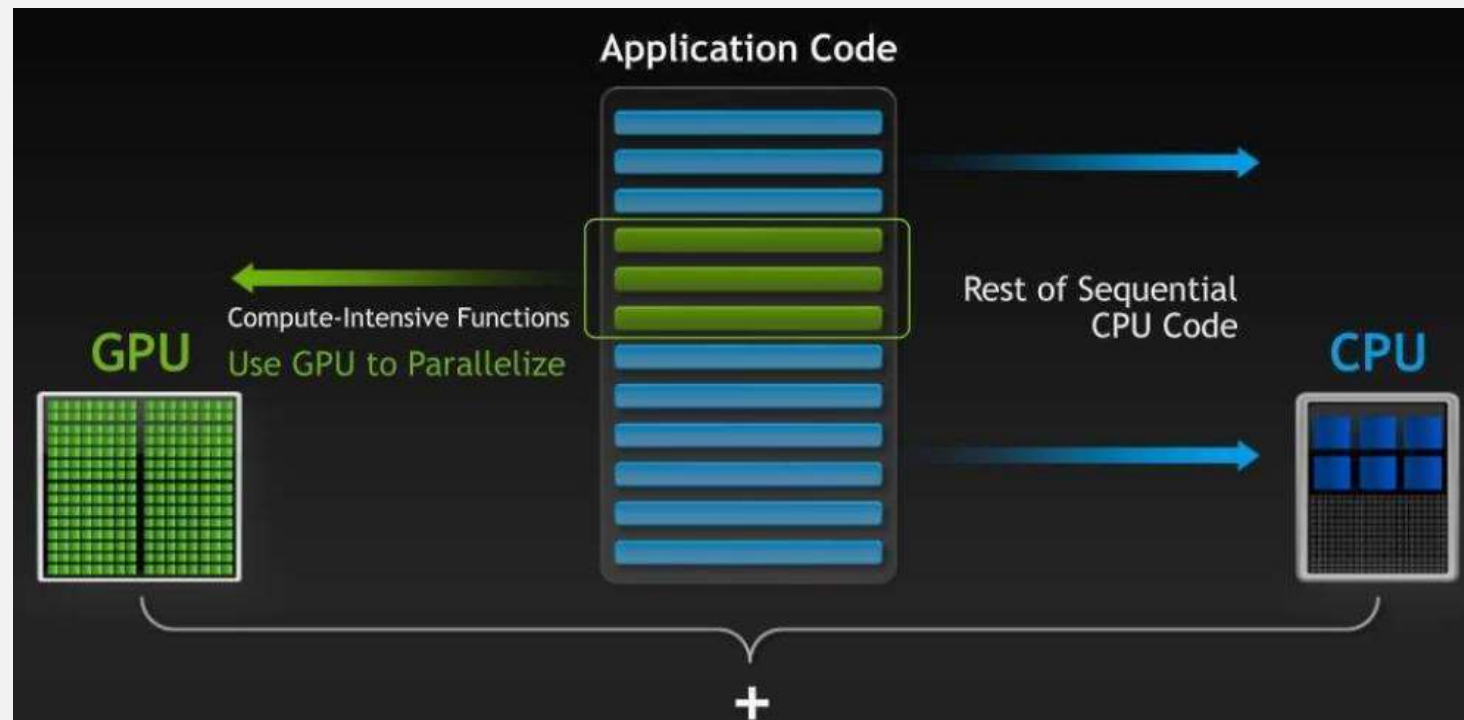
Relación CPU - GPU



Computación heterogénea

Introducción

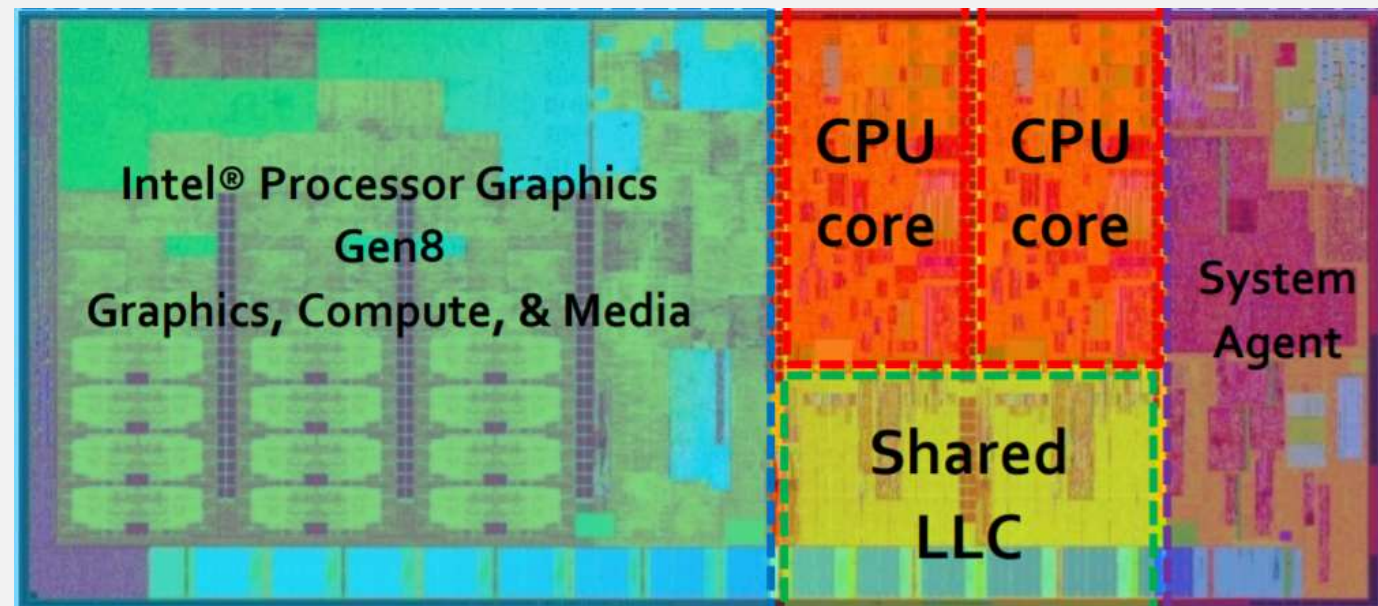
Relación CPU - GPU



Computación heterogénea

Introducción

GPU integrada.



Computación heterogénea

Introducción

GPU integrada.

Interna al procesador.

Facilidades de programación.
entornos Intel.

Ej.: Matriz 4000x4000
Secuencial: 836,77 seg.
OpenCL: 31,2 seg.



Computación heterogénea

OpenCL

Open Computing Language

Lenguaje sencillo para acceder a recursos heterogéneos de computación.

Compatible con APIs gráficas como OpenGL o OpenCV.

Dedicado a la ejecución en paralelo de tareas o para múltiples datos.

Aplicable a múltiples arquitecturas: GPU, CPU, CPU + GPU, múltiples GPUS, etc.



Computación heterogénea

OpenCL

Objetivos:

Utilizar todos los recursos hardware disponibles.

Modelo SIMD para la paralelización de datos.

Programación sencilla y potente.

Extensión de C. Eliminando cierta funcionalidad y extendiéndolo con operaciones vectoriales



Computación heterogénea

OpenCL

Partes de un programa en OpenCL:

Host: Ejecuta el código host, programa secuencial en la CPU principal. Utilizado para controlar la ejecución del kernel en el dispositivo, configurar los dispositivos y enviar y recibir información de los dispositivos.

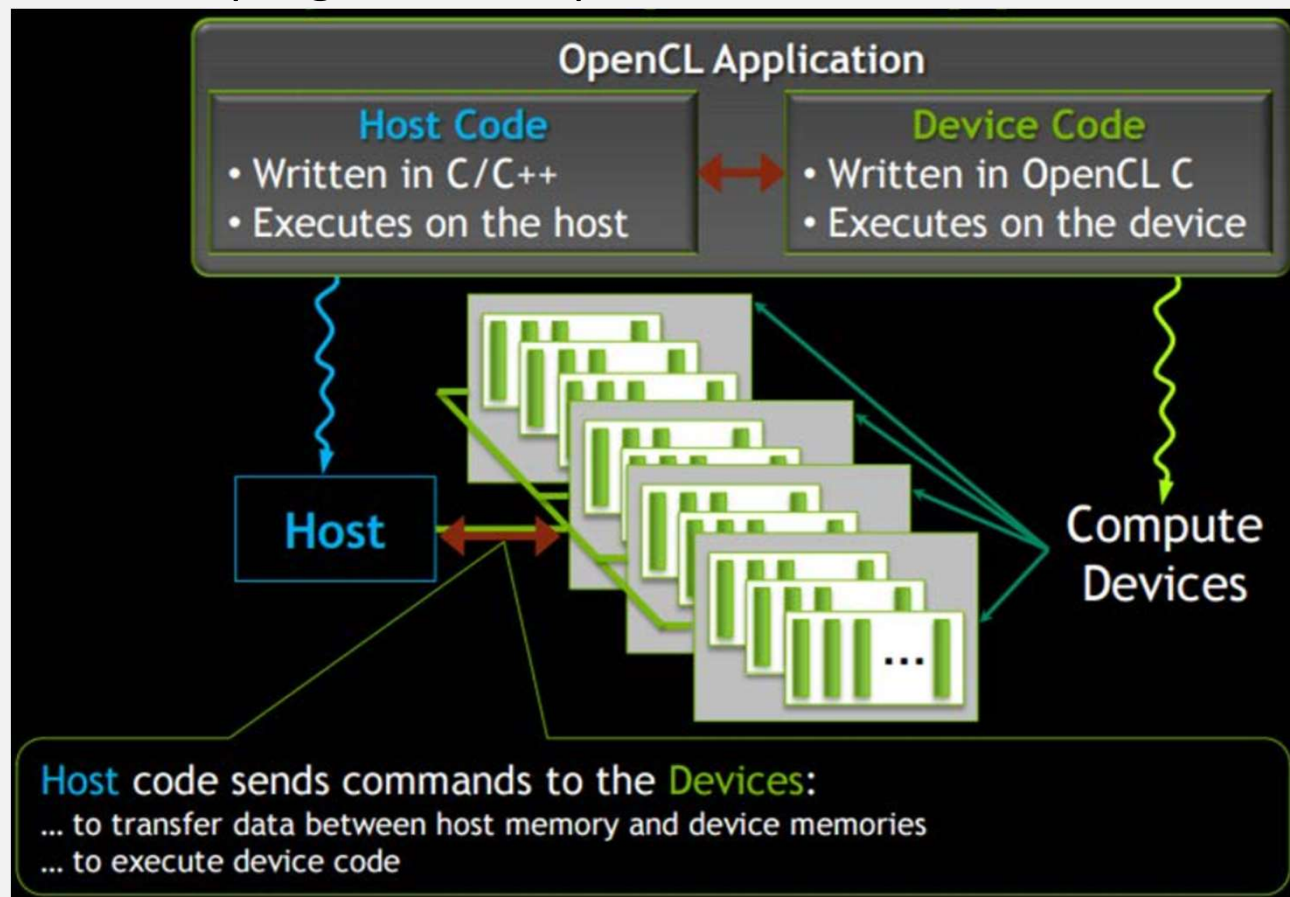
Dispositivo: Componente que contiene las unidades de procesamiento (vectorial o SIMD) y que ejecutarán el kernel, código que se ejecuta en paralelo. (GPU, CPU, FPGA, DSP).



Computación heterogénea

OpenCL

Partes de un programa en OpenCL:



Computación heterogénea

OpenCL

Modo de ejecución:

Host: Ejecuta el programa principal y envía tareas a los dispositivos para que éstos las procesen.

Es necesario crear un contexto en el que se incluyen los trabajos a ejecutar, los dispositivos que se utilizarán y la cola de comandos.



Computación heterogénea

OpenCL

Elementos en el desarrollo de programas OpenCl:

Contexto OpenCl: Se refiere al entorno de ejecución y contiene la información sobre los recursos necesarios para la ejecución de los kernels en los dispositivos.

Cola de Comandos: Permite al host para enviar comandos al dispositivo OpenCL. Los comandos se ejecutan en el orden en que se colocan en la cola.



Computación heterogénea

OpenCL

Tipos de datos específicos OpenCL.

`cl_platform_id`. Estructura que permite identificar una plataforma de OpenCL.

`cl_device_id`. Estructura que permite identificar un dispositivo.

`cl_context`. Permite identificar un contexto OpenCL.

`cl_command_queue`. Permite identificar una cola de comandos para poder administrarla y ejecutarla.

`cl_program`. Permite identificar un programa OpenCL (kernels o unidades de cómputo que se ejecutan en los dispositivos OpenCL)

`cl_kernel`. Tipo de dato para identificar un kernel OpenCL. Permite configurar, encolar y administrar la ejecución de kernels en dispositivos.

`cl_mem`. Identifica un objeto de memoria OpenCL, utilizados para almacenar datos que pueden ser accedidos por los kernels.



Computación heterogénea

OpenCL

Algunas funciones OpenCL.

clGetPlatformIDs

Se utiliza para obtener las plataformas OpenCL disponibles en el sistema.

Ejemplo de uso:

```
cl_platform_id platform;  
cl_int err;  
err = clGetPlatformIDs(1, &platform, NULL);  
if (err != CL_SUCCESS) { // Manejar el error }
```

Obtiene la plataforma 1 disponible y la almacena en la variable platform.



Computación heterogénea

OpenCL

Algunas funciones OpenCL.

clGetDeviceIDs.

Enumera y obtiene una lista de los dispositivos disponibles en una plataforma OpenCL específica.

```
cl_device_id devices[3];  
cl_int clGetDeviceIDs(cl_platform_id platform,  
                      cl_device_type device_type,  
                      cl_uint num_entries,  
                      cl_device_id *devices,  
                      cl_uint *num_devices);
```

Almacena en devices los identificadores de los dispositivos encontrados.



Computación heterogénea

OpenCL

Algunas funciones OpenCL.

clCreateContext

Crea un nuevo contexto OpenCL. Se le indica el dispositivo que ejecutará los comandos encolados en este contexto.

```
cl_context context;
```

```
cl_int err;
```

Crear un contexto asociado al dispositivo “device”.

```
context = clCreateContext(NULL, 1, &device, NULL, NULL, &err);
```



Computación heterogénea

OpenCL

Algunas funciones OpenCL.

clCreateCommandQueue

Crea una nueva cola de comandos asociada a un contexto OpenCL (context) y un dispositivo (device).

```
cl_command_queue queue;  
cl_int err;
```

```
// Crear una cola de comandos OpenCL  
queue = clCreateCommandQueue(context, device, 0, &err);  
if (err != CL_SUCCESS) { // Manejar el error }
```



Computación heterogénea

OpenCL

Algunas funciones OpenCL.

clCreateProgramWithSource

Crea un nuevo programa OpenCL a partir de código fuente en lenguaje OpenCL, contenido en la variable `kernelSource`. Se debe indicar el contexto (context) al que está asociado el programa.

```
program = clCreateProgramWithSource(context, 1, &kernelSource, NULL, &err);  
if (err != CL_SUCCESS) { // Manejar el error}
```



Computación heterogénea

OpenCL

Algunas funciones OpenCL.

clCreateKernel

Crea un nuevo kernel OpenCL a partir de un programa OpenCL previamente compilado y el nombre del kernel (my_kernel).

Una vez creado el kernel, se pueden configurar sus argumentos utilizando **clSetKernelArg** y encolar su ejecución utilizando **clEnqueueNDRangeKernel**.

```
cl_kernel kernel;  
cl_int err;  
kernel = clCreateKernel(program, my_kernel, &err);  
if (err != CL_SUCCESS) { // Manejar el error}
```



Computación heterogénea

OpenCL

Algunas funciones OpenCL.

clCreateBuffer

Se utiliza para crear un nuevo buffer de datos OpenCL. Se debe especificar el contexto OpenCL, el tipo de acceso al buffer (lectura o escritura del buffer por parte de la CPU), el tamaño del buffer en bytes y otros parámetros opcionales.

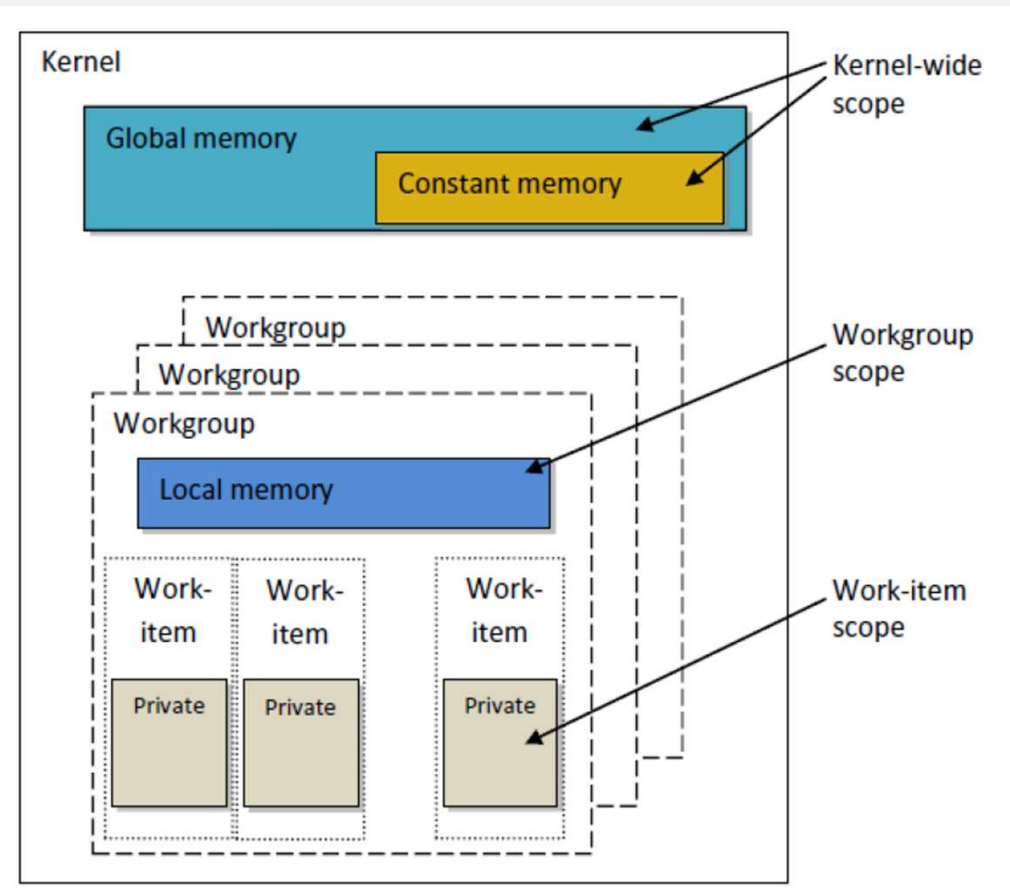
Funciones relacionadas: `clEnqueueReadBuffer`, `clEnqueueWriteBuffer` y `clSetKernelArg`.



Computación heterogénea

OpenCL

Manejo de memoria en OpenCL.



Computación heterogénea

OpenCL

Manejo de memoria en OpenCL.

La memoria global es accesible para todas las unidades de cómputo dentro del dispositivo y desde la memoria del host (CPU).

La memoria local es accesible por todo el grupo de trabajo, es decir, se comparte entre todas las unidades dentro del grupo y no es accesible por otros grupos.

La memoria privada es única para cada elemento de trabajo (work-ítem). Sólo es accesible para ese elemento y no se comparte entre elementos de trabajo



Computación heterogénea

OpenCL

Ejemplo.

Traditional loop as a function in C

```
void
trad_mul(int n,
         const float *a,
         const float *b,
         float *c)
{
    int i;
    for (i=0; i<n; i++)
        c[i] = a[i] * b[i];
}
```



OpenCL C kernel

```
__kernel void
dp_mul(__global const float *a,
       __global const float *b,
       __global float *c)
{
    int id = get_global_id(0);

    c[id] = a[id] * b[id];
} // execute over n "work items"
```